

A4 — Modelo Lógico Relacional e Justificativas de Mapeamento

1. Esquema relacional

Chaves primárias sublinhadas; chaves estrangeiras indicadas entre parênteses. Atributos que compõem chave primária composta estão marcados nas observações de cada tabela.

```
tb_user(id_user, user, password, email)

tb_seller(id_seller (fk -> tb_user), CNPJ)

tb_client(id_client (fk -> tb_user), phone)

tb_category(id_category, category, description, id_category_parent (fk -> tb_category))

tb_product(id_product, name, value, quantity,
id_seller (fk -> tb_seller), id_category (fk -> tb_category))

tb_supplier(id_supplier, name, CNPJ, phone)

tb_cart(id_cart, totalvalue, itemquantity, id_client (fk -> tb_client))

tb_buyItem(id_product (fk -> tb_product), id_cart (fk -> tb_cart), quantity, unit_value)

tb_order(id_order, date_order, totalvalue, id_client (fk -> tb_client))

tb_orderItem(id_order (fk -> tb_order), id_product (fk -> tb_product), quantity, unit_value)

tb_order_status(id_order (fk -> tb_order), date_status, status)

tb_product_supplier(id_product (fk -> tb_product), id_supplier (fk -> tb_supplier),
delivery_time)
```

Observação: tb_buyItem, tb_orderItem e tb_product_supplier têm chave primária composta pelas duas colunas de FK indicadas. tb_order_status (entidade fraca) tem chave primária composta por id_order + date_status.

2. Justificativas das decisões de mapeamento

2.1 Generalização/especialização: tb_user → tb_seller / tb_client

Criamos a tabela tb_user para guardar as informações que são comuns aos usuários, como usuário, senha e e-mail. Depois, separamos vendedor e cliente nas tabelas tb_seller e tb_client, pois cada um possui informações próprias.

- Não colocamos tudo em uma única tabela porque vendedor e cliente possuem dados diferentes. Por exemplo, o vendedor possui CNPJ e o cliente possui telefone. Se tudo ficasse em tb_user, alguns campos poderiam ficar vazios sem necessidade.
- Também não criamos vendedor e cliente totalmente separados porque isso faria repetir informações como usuário, senha e e-mail.
- Dessa forma, tb_seller e tb_client ficam ligados à tb_user. Um mesmo usuário também pode ser cliente e vendedor ao mesmo tempo.

- A especialização é parcial, pois um usuário pode existir sem ser vendedor ou cliente, e não exclusiva, pois ele pode ser os dois ao mesmo tempo.

2.2 Autorrelacionamento: tb_category.id_category_parent

Na tabela tb_category, usamos id_category_parent para permitir que uma categoria esteja ligada a outra categoria.

- Isso permite criar categorias e subcategorias. Uma categoria pode possuir várias subcategorias, mas cada subcategoria possui no máximo uma categoria acima dela.
- O campo id_category_parent pode ficar vazio (NULL), pois uma categoria principal não precisa estar ligada a outra categoria.

2.3 Entidade fraca: tb_order_status

A tabela tb_order_status depende da tabela tb_order, pois não existe um status sem um pedido relacionado.

- Por isso, usamos id_order junto com date_status como chave primária. Assim, conseguimos identificar cada mudança de status de um pedido.
- Não criamos um novo ID somente para o status porque essa tabela já pode ser identificada pelo pedido e pela data do status.
- Essa estrutura também permite guardar o histórico do pedido, mostrando os diferentes status que ele teve ao longo do tempo.

2.4 Relacionamentos N:N com atributo próprio

No modelo existem três relacionamentos N:N que foram transformados em tabelas associativas:

Associativa	Relação resolvida	Atributo(s) próprio(s)
tb_buyItem	carrinho ↔ produto	quantity, unit_value
tb_orderItem	pedido ↔ produto	quantity, unit_value
tb_product_supplier	produto ↔ fornecedor	delivery_time

Essas informações ficam nas tabelas associativas porque dependem das duas entidades relacionadas. Por exemplo, um produto pode aparecer em diferentes carrinhos com quantidades diferentes. Da mesma forma, o prazo de entrega de um produto pode mudar dependendo do fornecedor.

2.5 Chave natural vs. chave substituta

- Nas principais tabelas, como tb_user, tb_product, tb_category, tb_supplier, tb_cart e tb_order, usamos um ID como chave primária. Esses IDs são gerados automaticamente e servem para identificar cada registro.
- Informações como CNPJ e usuário podem ser únicas, mas não foram usadas como chave primária, pois podem precisar ser alteradas ou corrigidas.
- Nas tabelas tb_seller e tb_client, o ID também está ligado ao usuário da tabela tb_user, evitando criar outro identificador sem necessidade.

- Nas tabelas tb_buyItem, tb_orderItem e tb_product_supplier, usamos chaves compostas, formadas pela combinação dos IDs das tabelas relacionadas.
- Na tabela tb_order_status, usamos id_order junto com date_status para identificar cada registro.